

Методический материал для студентов

Computer Vision в Python

Как компьютер видит изображения

Фильтр Собеля

2 урока / 2 пары по 80 минут
Формат работы: Google Colab
Библиотеки: OpenCV, NumPy, Matplotlib

Краткая аннотация

Материал объясняет переход от классического машинного обучения на табличных данных к Computer Vision. Студенты повторяют Decision Tree и Random Forest, затем изучают изображение как массив чисел, RGB/BGR, grayscale, загрузку изображений в Google Colab и поиск границ с помощью фильтра Собеля.

Содержание

1. Место темы в курсе и цели обучения
2. 2. Урок 1: повторение Decision Tree и Random Forest
3. 3. Переход от табличных данных к изображениям
4. 4. Что такое Computer Vision
5. 5. Как компьютер хранит изображение
6. 6. Практика в Google Colab: загрузка и отображение изображения
7. 7. Урок 2: границы изображения и фильтр Собеля
8. 8. Полный код занятия с подробным разбором
9. 9. Практические задания на уроке
10. 10. Итоговые вопросы для закрепления
11. 11. Домашнее задание

1. Место темы в курсе и цели обучения

До этой темы студенты уже познакомились с базовыми методами машинного обучения: регрессией, классификацией, метриками качества, Decision Tree и Random Forest. Эти модели чаще всего изучаются на табличных данных, где каждый объект представлен строкой, а каждый признак - отдельной колонкой.

В Computer Vision ситуация другая: объектом становится изображение. Но для компьютера изображение также является данными. Главное отличие в том, что вместо привычной таблицы мы работаем с матрицами пикселей.

Элемент	Объяснение
Табличные данные	Строки и колонки: возраст, цена, посещаемость,

	оценка, площадь, количество комнат.
Изображение	Матрица пикселей. Каждый пиксель хранит информацию о яркости или цвете.
Computer Vision	Область AI, где компьютер анализирует изображения и видео.
Фильтр Собеля	Классический алгоритм для поиска границ на изображении.

Цели двух уроков

- Повторить смысл Decision Tree и Random Forest.
- Понять, почему изображение тоже является набором данных.
- Научиться читать изображение в Google Colab.
- Понять разницу между RGB, BGR и grayscale.
- Узнать, что такое пиксель и канал изображения.
- Научиться применять фильтр Собеля для поиска границ.
- Сравнить Sobel X, Sobel Y и объединённый Sobel.
- Понять, зачем границы важны для Computer Vision и будущих CNN-моделей.

Ожидаемый результат

После двух уроков студент должен уметь загрузить изображение в Google Colab, прочитать его через OpenCV, вывести оригинальное и grayscale-изображение, применить Sobel X и Sobel Y, объединить результаты и объяснить, какие границы были найдены.

План урока 1: 80 минут

Время	Блок	Что делает студент
0-10 мин	Введение и связь с прошлой темой	Вспоминает, какие модели уже изучались.
10-30 мин	Повторение Decision Tree и Random Forest	Отвечает на вопросы, разбирает логику дерева и леса.
30-45 мин	Переход к Computer Vision	Понимает, чем изображение отличается от таблицы.
45-60 мин	Пиксель, RGB, BGR, grayscale	Разбирает изображение как массив чисел.
60-80 мин	Google Colab: загрузка и отображение	Запускает код, загружает изображение, проверяет shape.

План урока 2: 80 минут

Время	Блок	Что делает студент
0-15 мин	Повторение изображения как массива	Объясняет shape, пиксели, grayscale.
15-30 мин	Границы изображения	Понимает, что граница - резкое изменение яркости.
30-50 мин	Фильтр Собеля	Разбирает Sobel X, Sobel Y и ядра 3x3.
50-65 мин	Код Sobel в Google Colab	Применяет фильтр к своему изображению.
65-75 мин	Эксперименты: ksize и blur	Сравнивает результаты.
75-80 мин	Итог и домашнее задание	Фиксирует основные выводы.

2. Урок 1. Повторение Decision Tree и Random Forest

2.1. Что такое Decision Tree

Decision Tree, или дерево решений, - это модель машинного обучения, которая принимает решение через последовательность вопросов. Каждый вопрос делит данные на две части. В конце дерево приходит к листу, где находится финальный ответ.

Дерево решений удобно объяснять через обычные условия if/else. Например, если мы хотим предсказать, сдаст ли студент экзамен, дерево может рассуждать так:

```
if practice_score > 60:
    result = "pass"
else:
    if attendance > 75:
        result = "pass"
    else:
        result = "fail"
```

Важно понимать: в обычном программировании эти условия пишет человек. В Decision Tree модель сама подбирает условия на основе данных.

Элемент	Объяснение
Узел	Точка в дереве, где задаётся вопрос.
Корень	Первый вопрос дерева.
Ветка	Переход после ответа на вопрос.
Лист	Финальный ответ дерева.
Глубина	Количество уровней вопросов.

2.2. Пример данных для дерева решений

```
import pandas as pd

data = {
    "attendance": [95, 60, 80, 50, 90, 55],
    "homework_done": [1, 0, 1, 0, 1, 0],
    "practice_score": [88, 45, 70, 40, 92, 35],
    "result": ["pass", "fail", "pass", "fail", "pass", "fail"]
}

df = pd.DataFrame(data)
df
```

В этом примере каждая строка - один студент. Колонки attendance, homework_done и practice_score являются признаками. Колонка result является целевой переменной.

Элемент	Объяснение
attendance	Посещаемость студента в процентах.
homework_done	Выполнял ли студент домашние задания: 1 - да, 0 - нет.
practice_score	Оценка за практику.
result	Итог: pass или fail.

2.3. Как дерево выбирает лучший вопрос

Дерево стремится разделить данные так, чтобы в каждой группе стало меньше смешения классов. Если до вопроса в группе были pass и fail вместе, то после хорошего вопроса группы должны стать более чистыми.

Например, до разделения:

```
pass, pass, fail, fail, pass, fail
```

После хорошего разделения:

```
Левая группа: fail, fail, fail
```

```
Правая группа: pass, pass, pass
```

Это хорошее разделение, потому что классы почти не перемешаны.

Главная мысль

Decision Tree ищет такие вопросы, которые делают группы объектов более чистыми. Чем чище группы после разделения, тем полезнее вопрос.

2.4. Impurity простыми словами

Impurity можно перевести как загрязнённость. Узел считается загрязнённым, если в нём перемешаны разные классы.

Чистый узел:

```
pass, pass, pass, pass
```

Грязный узел:

```
pass, fail, pass, fail
```

Дерево строится так, чтобы impurity уменьшалась после каждого разделения.

2.5. Проблема переобучения Decision Tree

Decision Tree легко переобучается. Это происходит, когда дерево становится слишком глубоким и начинает запоминать частные случаи из обучающего набора.

Переобучение

Переобучение - это ситуация, когда модель очень хорошо работает на тренировочных данных, но хуже работает на новых данных, которых она раньше не видела.

Если не ограничивать дерево, оно может построить слишком много условий:

```
Если score <= 73.5
Если age <= 21.5
Если attendance <= 92.5
Если homework_done <= 0.5
Если practice_score <= 67.5
...
```

Такое дерево может выглядеть умным, но на самом деле оно может просто запоминать обучающие примеры.

```

from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=None,
    random_state=42
)

model.fit(X_train, y_train)

```

Значение `max_depth=None` означает, что глубина дерева не ограничена. Такое дерево может стать слишком сложным.

2.6. Как ограничить сложность дерева

```

model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

```

Параметр `max_depth=3` означает, что дерево сможет задать не больше трёх уровней вопросов. Это снижает риск переобучения.

Элемент	Объяснение
<code>max_depth</code> маленький	Модель простая. Есть риск недообучения.
<code>max_depth</code> большой	Модель сложная. Есть риск переобучения.
<code>max_depth</code> подобран разумно	Модель лучше обобщает закономерности.

2.7. Что такое Random Forest

Random Forest - это модель, которая состоит не из одного дерева, а из множества деревьев. Каждое дерево делает своё предсказание, а итоговый ответ выбирается голосованием.

```

Tree 1 -> pass
Tree 2 -> fail
Tree 3 -> pass
Tree 4 -> pass
Tree 5 -> pass

Итого Random Forest -> pass

```

Если Decision Tree - это один эксперт, то Random Forest - это группа экспертов. Один эксперт может ошибиться, но решение группы обычно стабильнее.

```

from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

forest.fit(X_train, y_train)

```

Элемент	Объяснение
<code>n_estimators=100</code>	Количество деревьев в лесу.
<code>max_depth=5</code>	Максимальная глубина каждого дерева.
<code>random_state=42</code>	Фиксация случайности для повторяемого результата.

3. Переход от табличных данных к изображениям

В Decision Tree и Random Forest мы работали с готовыми признаками. Например, в задаче про студентов признаки уже были понятны: attendance, homework_done, practice_score.

Но если мы хотим анализировать изображение, готовых колонок у нас нет. Фотография не содержит колонку "форма объекта", "количество границ" или "цвет объекта". Сначала нужно понять, как изображение хранится внутри компьютера.

Табличная задача:

[attendance, homework_done, practice_score] -> модель -> pass/fail

Computer Vision:

изображение -> пиксели -> признаки -> модель -> предсказание

На этом уроке мы ещё не обучаем нейросеть. Мы изучаем базовый фундамент: как читать изображение, как увидеть его как массив чисел и как найти простые признаки - границы.

4. Что такое Computer Vision

Computer Vision, или компьютерное зрение, - это область искусственного интеллекта, где компьютер анализирует изображения и видео.

- Классификация изображения: определить, что изображено на фото.
- Детекция объектов: найти объект и выделить его рамкой.
- Сегментация: разделить изображение на области.
- Распознавание лиц: найти и идентифицировать лицо.
- OCR: распознать текст на изображении.
- Анализ медицинских снимков: находить признаки заболеваний.
- Контроль качества на производстве: искать дефекты деталей.

4.1. Примеры применения Computer Vision

Элемент	Объяснение
Face ID	Система распознаёт лицо владельца телефона.
Беспилотные автомобили	Камеры анализируют дорогу, пешеходов, машины, знаки и разметку.
Медицина	AI помогает анализировать снимки и выделять подозрительные области.
Маркетплейсы	Поиск похожих товаров по фотографии.
Безопасность	Камеры могут обнаруживать движение, людей и транспорт.
Производство	Система может находить трещины, царапины и дефекты.

5. Как компьютер хранит изображение

5.1. Пиксель

Пиксель - это маленькая точка изображения. Любая цифровая картинка состоит из пикселей. Если приблизить изображение очень сильно, можно увидеть отдельные квадратики - это и есть пиксели.

Например, размер изображения 1920 x 1080 означает:

```
1920 пикселей по ширине
1080 пикселей по высоте

Всего пикселей:
1920 * 1080 = 2 073 600
```

То есть Full HD изображение содержит больше двух миллионов пикселей.

5.2. Grayscale изображение

Grayscale - это изображение в оттенках серого. В таком изображении каждый пиксель хранится одним числом.

```
0    -> чёрный
255  -> белый
128  -> серый
```

Пример маленького grayscale-изображения 5x5:

```
[
  [0, 0, 0, 0, 0],
  [0, 255, 255, 255, 0],
  [0, 255, 100, 255, 0],
  [0, 255, 255, 255, 0],
  [0, 0, 0, 0, 0]
]
```

Здесь каждое число показывает яркость конкретного пикселя.

5.3. Цветное изображение RGB

Цветное изображение обычно хранится через три канала: Red, Green, Blue. Это называется RGB.

```
R -> Red    -> красный
G -> Green   -> зелёный
B -> Blue    -> синий
```

Каждый пиксель цветного изображения хранит три числа.

```
[255, 0, 0]    -> красный
[0, 255, 0]    -> зелёный
[0, 0, 255]    -> синий
[255, 255, 255] -> белый
[0, 0, 0]      -> чёрный
```

5.4. Важная особенность OpenCV: BGR вместо RGB

OpenCV читает цветные изображения не в формате RGB, а в формате BGR. Это значит, что порядок каналов другой: Blue, Green, Red.

```
RGB: [Red, Green, Blue]
BGR: [Blue, Green, Red]
```

Поэтому если прочитать изображение через OpenCV и сразу показать его через Matplotlib, цвета могут быть неправильными. Для корректного отображения нужно конвертировать BGR в RGB.

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

Типичная ошибка

Студент загружает картинку, читает её через `cv2.imread` и сразу показывает через `plt.imshow(image)`. В результате цвета могут выглядеть странно. Нужно использовать `cv2.cvtColor(image,`

```
cv2.COLOR_BGR2RGB).
```

6. Практика в Google Colab: загрузка и отображение изображения

6.1. Импорт библиотек

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files
```

Элемент	Объяснение
cv2	OpenCV. Библиотека для работы с изображениями и видео.
numpy	Библиотека для работы с массивами чисел.
matplotlib.pyplot	Библиотека для визуализации графиков и изображений.
files	Модуль Google Colab для загрузки файлов с компьютера.

6.2. Загрузка изображения

В Google Colab важно не писать имя файла вручную, потому что Colab может переименовать файл при загрузке. Например, если файл называется image 2.png, Colab может сохранить его как image 2 (1).png.

```
uploaded = files.upload()

filename = list(uploaded.keys())[0]
print("Загруженный файл:", filename)
```

Код `list(uploaded.keys())[0]` берёт реальное имя файла, которое Colab сохранил после загрузки.

6.3. Чтение изображения через OpenCV

```
image = cv2.imread(filename)

if image is None:
    print("Ошибка: изображение не прочиталось")
else:
    print("Изображение успешно загружено")
    print("Размер изображения:", image.shape)
```

Если `cv2.imread` не смог прочитать файл, он возвращает `None`. Поэтому перед использованием `image.shape` нужно обязательно проверить `image is None`.

Ошибка `AttributeError`

Если написать `print(image.shape)`, когда `image` равен `None`, возникнет ошибка: `AttributeError: NoneType object has no attribute shape`. Это значит, что изображение не было прочитано.

6.4. Почему возникает ошибка NoneType

- Файл называется не так, как написано в коде.
- В имени файла есть пробелы, например image 2.png.
- Colab переименовал файл, например image 2 (4).png.
- Файл не загрузился.
- Формат файла повреждён или не поддерживается.

6.5. Правильное отображение изображения

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6, 6))
plt.imshow(image_rgb)
plt.axis("off")
plt.title("Original image")
plt.show()
```

Элемент	Объяснение
cv2.cvtColor	Меняет цветовое пространство изображения.
cv2.COLOR_BGR2RGB	Конвертация из BGR в RGB.
plt.figure	Создаёт область отображения.
plt.imshow	Показывает изображение.
plt.axis("off")	Убирает оси координат.
plt.show()	Выводит изображение на экран.

6.6. Анализ размера изображения

```
print(image.shape)
```

Например, результат может быть таким:

```
(720, 1280, 3)
```

Элемент	Объяснение
720	Высота изображения в пикселях.
1280	Ширина изображения в пикселях.
3	Количество цветовых каналов.

6.7. Просмотр одного пикселя

```
pixel = image[0, 0]
print(pixel)
```

Например, результат:

```
[34 58 120]
```

В OpenCV это BGR, то есть:

```
Blue   = 34
Green  = 58
Red    = 120
```

6.8. Перевод изображения в grayscale

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print(gray.shape)
```

```
plt.figure(figsize=(6, 6))
plt.imshow(gray, cmap="gray")
plt.axis("off")
plt.title("Grayscale image")
plt.show()
```

После перевода в grayscale shape обычно содержит только два числа, например:

```
(720, 1280)
```

Это означает, что теперь у изображения нет трёх цветовых каналов. Осталась только одна матрица яркости.

7. Урок 2. Границы изображения и фильтр Собеля

7.1. Что такое граница на изображении

Граница на изображении - это место, где значения пикселей резко меняются. Чаще всего это переход от светлой области к тёмной или от одного объекта к другому.

Пример строки пикселей:
[10, 10, 10, 10, 200, 200, 200]

Резкое изменение находится между 10 и 200.

Для человека это может быть край объекта. Для компьютера это резкий скачок чисел.

7.2. Почему границы важны

Границы помогают понять форму объекта. Даже если убрать цвет и оставить только контуры, человек часто всё равно понимает, что изображено. В Computer Vision границы являются одним из базовых признаков.

- Границы помогают выделить форму объекта.
- Контуры помогают отделить объект от фона.
- По границам можно находить линии, углы и текстуры.
- Многие современные нейросети на первых слоях тоже учатся находить простые границы и линии.

7.3. Что такое фильтр изображения

Фильтр - это небольшая матрица, которая проходит по изображению и выполняет математическую операцию с пикселями. Фильтр может размывать изображение, усиливать резкость, находить границы или убирать шум.

- Blur-фильтр размывает изображение.
- Sharpen-фильтр усиливает резкость.
- Sobel-фильтр ищет границы.
- GaussianBlur помогает уменьшить шум.

7.4. Фильтр как маленькое окно

Фильтр обычно имеет небольшой размер, например 3x3. Он берёт маленький участок изображения, умножает значения пикселей на значения фильтра и получает новое число.

Участок изображения 3x3:
10 10 10

```
10 200 200
10 200 200
```

Фильтр 3x3 проходит по таким участкам по всему изображению.

7.5. Что такое фильтр Собеля

Фильтр Собеля - это классический алгоритм для поиска границ. Он оценивает, насколько резко меняется яркость изображения по горизонтали и вертикали.

Обычно используют два варианта:

Элемент	Объяснение
Sobel X	Ищет изменения по оси X. Хорошо выделяет вертикальные границы.
Sobel Y	Ищет изменения по оси Y. Хорошо выделяет горизонтальные границы.

7.6. Матрица Sobel X

```
[ -1  0  1]
[ -2  0  2]
[ -1  0  1]
```

Sobel X реагирует на изменения слева направо. Если слева тёмная область, а справа светлая, фильтр даст сильный отклик. Поэтому Sobel X хорошо выделяет вертикальные края.

7.7. Матрица Sobel Y

```
[ -1 -2 -1]
[  0  0  0]
[  1  2  1]
```

Sobel Y реагирует на изменения сверху вниз. Если сверху тёмная область, а снизу светлая, фильтр даст сильный отклик. Поэтому Sobel Y хорошо выделяет горизонтальные края.

7.8. Почему Sobel часто применяют к grayscale

Sobel ищет изменение яркости. Для поиска границ не всегда нужен цвет. Поэтому изображение часто сначала переводят в grayscale, а затем применяют Sobel.

```
Цветное изображение:
height x width x 3
```

```
Grayscale изображение:
height x width
```

Работать с одной матрицей яркости проще и быстрее, чем с тремя цветовыми каналами.

8. Полный код занятия с подробным разбором

Ниже приведён полный код, который можно запустить в Google Colab. Он загружает изображение, показывает оригинал, переводит его в grayscale, применяет Sobel X, Sobel Y, объединяет результаты и показывает Sobel после размытия.

```
import cv2
import numpy as np
```

```

import matplotlib.pyplot as plt

from google.colab import files

# 1. Загружаем изображение с компьютера
uploaded = files.upload()

# 2. Получаем реальное имя загруженного файла
filename = list(uploaded.keys())[0]
print("Загруженный файл:", filename)

# 3. Читаем изображение через OpenCV
image = cv2.imread(filename)

# 4. Проверяем, что изображение прочиталось
if image is None:
    print("Ошибка: изображение не прочиталось")
else:
    print("Изображение успешно загружено")
    print("Размер изображения:", image.shape)

# 5. Конвертируем BGR в RGB для корректного отображения
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 6. Переводим изображение в grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 7. Применяем Sobel по X
sobel_x = cv2.Sobel(
    gray,
    cv2.CV_64F,
    1,
    0,
    ksize=3
)

# 8. Применяем Sobel по Y
sobel_y = cv2.Sobel(
    gray,
    cv2.CV_64F,
    0,
    1,
    ksize=3
)

# 9. Переводим результат в формат для отображения
sobel_x_abs = cv2.convertScaleAbs(sobel_x)
sobel_y_abs = cv2.convertScaleAbs(sobel_y)

# 10. Объединяем Sobel X и Sobel Y
sobel_combined = cv2.addWeighted(
    sobel_x_abs,
    0.5,
    sobel_y_abs,
    0.5,
    0
)

# 11. Размываем grayscale-изображение

```

```

blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# 12. Применяем Sobel к размытому изображению
sobel_x_blur = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobel_y_blur = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)

sobel_x_blur_abs = cv2.convertScaleAbs(sobel_x_blur)
sobel_y_blur_abs = cv2.convertScaleAbs(sobel_y_blur)

sobel_blur_combined = cv2.addWeighted(
    sobel_x_blur_abs,
    0.5,
    sobel_y_blur_abs,
    0.5,
    0
)

# 13. Отображаем все результаты
plt.figure(figsize=(18, 12))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(sobel_x_abs, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(sobel_y_abs, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Combined Sobel")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(sobel_blur_combined, cmap="gray")
plt.title("Sobel after GaussianBlur")
plt.axis("off")

plt.show()

```

8.1. Подробный разбор кода

Элемент	Объяснение
<code>files.upload()</code>	Открывает окно загрузки файла в Google Colab.
<code>list(uploaded.keys())[0]</code>	Получает настоящее имя загруженного файла.
<code>cv2.imread(filename)</code>	Читает изображение через OpenCV.
<code>image is None</code>	Проверяет, удалось ли прочитать изображение.

image.shape	Показывает высоту, ширину и количество каналов.
cv2.COLOR_BGR2RGB	Исправляет порядок каналов для Matplotlib.
cv2.COLOR_BGR2GRAY	Переводит изображение в оттенки серого.
cv2.Sobel(gray, cv2.CV_64F, 1, 0)	Применяет Sobel по X.
cv2.Sobel(gray, cv2.CV_64F, 0, 1)	Применяет Sobel по Y.
cv2.convertScaleAbs	Берёт абсолютные значения и переводит результат в удобный формат.
cv2.addWeighted	Смешивает два изображения с заданными весами.
cv2.GaussianBlur	Размывает изображение для уменьшения шума.

8.2. Почему используется cv2.CV_64F

После применения Sobel значения могут быть отрицательными. Например, если переход идёт от светлого к тёмному, результат может иметь один знак, а если от тёмного к светлому - другой. Поэтому используется тип cv2.CV_64F, который позволяет хранить такие значения без потери информации.

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

После этого для отображения результата используется:

```
sobel_x_abs = cv2.convertScaleAbs(sobel_x)
```

Эта функция переводит результат в изображение, которое можно нормально показать через Matplotlib.

8.3. Объединение Sobel X и Sobel Y

```
sobel_combined = cv2.addWeighted(
    sobel_x_abs,
    0.5,
    sobel_y_abs,
    0.5,
    0
)
```

Здесь мы берём 50% результата Sobel X и 50% результата Sobel Y. В итоге получаем изображение, где видны и вертикальные, и горизонтальные границы.

8.4. Альтернативный способ объединения: magnitude

Можно объединить Sobel X и Sobel Y через математическую формулу общей силы границы:

```
sobel_magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_magnitude = np.uint8(np.clip(sobel_magnitude, 0, 255))

plt.figure(figsize=(8, 8))
plt.imshow(sobel_magnitude, cmap="gray")
plt.title("Sobel magnitude")
plt.axis("off")
plt.show()
```

Формула выглядит так:

```
magnitude = sqrt(SobelX^2 + SobelY^2)
```

Она показывает общую силу границы, учитывая изменения и по X, и по Y.

8.5. Эксперимент с ksize

Параметр ksize отвечает за размер ядра фильтра. Можно попробовать значения 3, 5 и 7.

```

for k in [3, 5, 7]:
    sobel_x_k = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=k)
    sobel_y_k = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=k)

    sobel_x_k_abs = cv2.convertScaleAbs(sobel_x_k)
    sobel_y_k_abs = cv2.convertScaleAbs(sobel_y_k)

    sobel_k_combined = cv2.addWeighted(
        sobel_x_k_abs,
        0.5,
        sobel_y_k_abs,
        0.5,
        0
    )

    plt.figure(figsize=(6, 6))
    plt.imshow(sobel_k_combined, cmap="gray")
    plt.title(f"Sobel with ksize={k}")
    plt.axis("off")
    plt.show()

```

После запуска нужно сравнить, как изменились границы. Обычно при большем ksize изображение становится более сглаженным, а мелкий шум может стать менее заметным.

8.6. Зачем использовать GaussianBlur перед Sobel

Sobel реагирует на резкие изменения яркости. Но шум на изображении - это тоже резкие изменения. Поэтому Sobel может выделить слишком много лишних мелких деталей. Чтобы уменьшить шум, перед Sobel часто применяют размытие.

```
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

Элемент	Объяснение
gray	Исходное grayscale-изображение.
(5, 5)	Размер окна размытия.
0	Стандартное отклонение. OpenCV подберёт его автоматически.

9. Практические задания на уроке

Практика 1. Загрузка и анализ изображения

Задача: загрузить любое изображение в Google Colab и вывести базовую информацию.

12. Загрузите изображение через `files.upload()`.
13. Получите настоящее имя файла через `list(uploaded.keys())[0]`.
14. Прочитайте изображение через `cv2.imread()`.
15. Проверьте, что `image` is not `None`.
16. Выведите `image.shape`.
17. Выведите значение одного пикселя `image[0, 0]`.

Практика 2. RGB/BGR и grayscale

18. Покажите изображение без конвертации BGR в RGB.
19. Покажите изображение после `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`.
20. Сравните результат.

21. Переведите изображение в grayscale.
22. Выведите shape цветного и grayscale-изображения.

Практика 3. Sobel X и Sobel Y

23. Примените Sobel X.
24. Примените Sobel Y.
25. Покажите оба результата.
26. Ответьте, какие границы лучше видны на Sobel X.
27. Ответьте, какие границы лучше видны на Sobel Y.

Практика 4. Объединённый Sobel

28. Объедините Sobel X и Sobel Y через cv2.addWeighted().
29. Покажите оригинал, grayscale, Sobel X, Sobel Y и Combined Sobel на одном экране.
30. Сделайте вывод: где контуры видны лучше всего.

Практика 5. GaussianBlur перед Sobel

31. Примените GaussianBlur к grayscale-изображению.
32. Примените Sobel к размытому изображению.
33. Сравните Sobel без размытия и Sobel после размытия.
34. Опишите, стало ли меньше шума.

10. Итоговые вопросы для закрепления

35. Что такое Computer Vision?
36. Почему изображение можно считать данными?
37. Что такое пиксель?
38. Что означает image.shape для цветного изображения?
39. Чем RGB отличается от BGR?
40. Почему OpenCV использует BGR?
41. Что такое grayscale?
42. Почему Sobel часто применяют к grayscale?
43. Что такое граница на изображении?
44. Какие границы лучше выделяет Sobel X?
45. Какие границы лучше выделяет Sobel Y?
46. Зачем объединять Sobel X и Sobel Y?
47. Зачем использовать GaussianBlur перед Sobel?
48. Почему Sobel сам по себе не является нейросетью?
49. Как Sobel связан с будущей темой CNN?

11. Главные выводы по двум урокам

- Decision Tree работает через последовательность вопросов.
- Random Forest использует много деревьев и голосование.
- Табличные модели работают с готовыми признаками.
- В Computer Vision признаки часто нужно извлекать из изображения.
- Изображение для компьютера - это массив чисел.
- Цветное изображение обычно имеет высоту, ширину и 3 канала.
- OpenCV читает изображение в формате BGR.

- Для корректного отображения через Matplotlib нужно переводить BGR в RGB.
- Grayscale-изображение содержит одну матрицу яркости.
- Граница - это резкое изменение яркости.
- Sobel X выделяет вертикальные границы.
- Sobel Y выделяет горизонтальные границы.
- GaussianBlur помогает уменьшить шум перед поиском границ.
- Sobel показывает идею ручного извлечения признаков.
- В CNN нейросеть учится извлекать признаки автоматически.

12. Домашнее задание

Домашнее задание выполняется в Google Colab. Нужно создать отдельный notebook и сдать ссылку на него или файл .ipynb.

Тема домашнего задания

Анализ изображений с помощью фильтра Собеля.

Что нужно сделать

Выберите 3 разных изображения:

- 50. Портрет человека или фото лица.
- 51. Фото здания, улицы или города.
- 52. Фото предмета на столе или любого объекта крупным планом.

Для каждого изображения выполнить

- 53. Загрузить изображение в Google Colab.
- 54. Получить настоящее имя файла через `list(uploaded.keys())[0]`.
- 55. Прочитать изображение через `cv2.imread()`.
- 56. Проверить, что изображение успешно прочиталось.
- 57. Вывести размер изображения через `image.shape`.
- 58. Показать оригинальное изображение с правильными цветами.
- 59. Перевести изображение в grayscale.
- 60. Показать grayscale-изображение.
- 61. Применить Sobel X.
- 62. Применить Sobel Y.
- 63. Объединить Sobel X и Sobel Y.
- 64. Применить GaussianBlur перед Sobel.
- 65. Сравнить результат Sobel без размытия и после размытия.
- 66. Написать вывод по каждому изображению.

Обязательные выводы по каждому изображению

- 67. Какие границы лучше всего нашёл Sobel?
- 68. Что лучше видно на Sobel X?
- 69. Что лучше видно на Sobel Y?
- 70. Где Sobel сработал плохо?
- 71. Что изменилось после GaussianBlur?
- 72. Стало ли меньше шума?
- 73. Какое из трёх изображений лучше всего подходит для Sobel и почему?
- 74. Почему поиск границ важен для Computer Vision?

Требования к notebook

- Notebook должен быть аккуратно оформлен.
- Перед каждым блоком кода должен быть короткий markdown-комментарий.
- Код должен запускаться сверху вниз без ошибок.
- Все изображения результатов должны быть видны в notebook.
- В конце должен быть общий вывод на 5-8 предложений.

Критерии оценивания: 12 баллов

Критерий	Описание	Баллы
Загрузка изображений	Все 3 изображения корректно загружаются и читаются через OpenCV.	2
Original + grayscale	Для каждого изображения показан оригинал и grayscale.	2
Sobel X и Sobel Y	Для каждого изображения применены оба фильтра.	2
Combined Sobel	Sobel X и Sobel Y объединены и результат отображён.	2
GaussianBlur + Sobel	Есть сравнение результата до и после размытия.	2
Выводы	Есть осмысленные текстовые выводы по каждому изображению и общий вывод.	2

Минимальный шаблон для домашней работы

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]

image = cv2.imread(filename)

if image is None:
    print("Ошибка чтения изображения")
else:
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

    sobel_x_abs = cv2.convertScaleAbs(sobel_x)
    sobel_y_abs = cv2.convertScaleAbs(sobel_y)

    sobel_combined = cv2.addWeighted(sobel_x_abs, 0.5, sobel_y_abs, 0.5, 0)

    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    sobel_x_blur = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y_blur = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)

    sobel_x_blur_abs = cv2.convertScaleAbs(sobel_x_blur)
    sobel_y_blur_abs = cv2.convertScaleAbs(sobel_y_blur)
    sobel_blur_combined = cv2.addWeighted(sobel_x_blur_abs, 0.5, sobel_y_blur_abs, 0.5, 0)

    plt.figure(figsize=(18, 12))

    plt.subplot(2, 3, 1)
    plt.imshow(image_rgb)
    plt.title("Original")
    plt.axis("off")
```

```
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(sobel_x_abs, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(sobel_y_abs, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Combined Sobel")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(sobel_blur_combined, cmap="gray")
plt.title("Sobel after GaussianBlur")
plt.axis("off")

plt.show()
```

Финальный общий вывод для домашней работы

В конце notebook студент должен написать общий вывод. В выводе нужно объяснить, почему изображение является массивом чисел, как Sobel помогает находить границы, чем Sobel X отличается от Sobel Y и почему размытие может улучшать результат.